Campus Supply Desk - Assignment 2

Mailles Machiana | COMP3415 | November 8, 2026

Repository: https://github.com/McMerger/COMP3415-A2-Campus-Supply-Desk

1. How the program works

Campus Supply Desk is a Python program for ordering campus merchandise. It runs in the terminal and lets someone check stock, get a quote, place an order, view past orders and cancel an order. It stores prices as whole cents and saves stock and orders in a JSON file.

The entry point is `supplydesk/__main__.py`. It calls `cli.py`, which either runs a single command or opens the menu in `menu.py`. Both use `OrderService` in `service.py` for the actual calculations and order changes. `storage.py` handles the save file. A new session gets its starting products from `catalog.json`.

A quote checks stock but does not reserve anything. Placing the order checks again, reduces stock and saves the order with a new ID. Cancelling should return the items to stock once, while keeping the order and its original prices in history. The functional specification was used to decide the expected behavior, since the starter code does not always follow those rules.

2. Problems in the starter code

The references below are to `supplydesk/service.py` in the original commit, `82ed75e`. These are findings from the work after Checkpoint A.

Repeated quote and order code

Lines 15-60 and 65-112 repeat most of the same validation and pricing steps. Updating both copies makes it easy for quotes and orders to disagree. The shared parts now sit in `_build_quote`, which both operations call.

Student discount at exactly $50

Lines 48 and 98 use `> 5000`, so a student buying two $25 bags gets no discount. The requirement says at least $50. Changing this to `>=` gives the correct $5 reduction. Tests check this for both quotes and placed orders.

Duplicate products can oversell stock

Lines 36 and 86 check each line separately. With 20 notebooks in stock, lines of 11 and 10 both pass even though together they need 21. The fix totals the requested quantities for each product before changing stock.

Cancelling twice adds stock twice

Lines 142-147 return items to stock on every cancellation. Ordering two mugs and cancelling twice increases stock from 8 to 10. The method now returns immediately if the order is already cancelled.

Repeated lookup and stock loops

The order search is repeated in lines 128-141, and the stock loops in lines 114-117 and 142-145 do almost the same job in opposite directions. `_find_order` and `_change_stock` keep those operations in one place.

Unexplained pricing numbers

Values such as 5000, 10000 and 600 appear in both pricing methods. Named constants now make the discount threshold, free-delivery threshold and fees easier to find.

3. Changes and testing

The refactoring and bug fixes were done separately. First, the common logic was moved into helpers without changing the rules. The same six failing assertions remained. Then the three bugs were corrected, and all 27 tests at that stage passed. That separation helps show which changes reorganized the code and which ones changed its behavior.

The tests focus on amounts and saved data, where a small mistake has a noticeable result. They use temporary files so that runs do not change normal orders. Expected prices are worked out from the specification rather than calculated with the same production helper being tested.

Stage	Result	Evidence file in evidence/
Original starter tests	4 pass	01-starter-tests.txt
Added baseline tests	27 tests, 6 failures	02-regressions-before.txt
Refactoring before fixes	Same 6 failures	03-refactor-before-fixes.txt
Baseline fixes	27 pass	04-baseline-fixes.txt
Express tests before implementation	49 tests, 5 failures and 8 errors	05-cr01-before.txt
Completed version	49 pass	07-final-tests.txt

The earlier failures show what the new tests caught before implementation.

Important checks include the exact $50 discount boundary, delivery below and at $100 after discount, invalid quantity types, duplicate items and repeated cancellation. Other tests check that rejected orders do not change the save file or use an ID, and that reopening the program preserves stock and order history.

Real command processes and scripted menu input check output, errors, defaults and confirmations. Seven deliberately introduced mistakes were caught in separate copies. Their output is in `08-targeted-mutations.json`.

4. CR-01: Express delivery

Express was added to the service, the command options and the menu prompt. It always costs $12, even on an order that would qualify for free standard delivery. The student discount still applies only to merchandise. Regular customer and pickup remain the defaults, and the README and command help describe express.

Request	Expected and observed total
Regular, two notebooks, express	$22.00
Student, two hoodies, express	$84.00
Regular, three hoodies, standard delivery	$120.00
Regular, three hoodies, express	$132.00

Express tests compare quotes and orders, reopen saves and cancel orders. Original items and totals stay recorded, stock is restored once, and IDs are not reused. Unsupported fulfillment values still raise an error.

5. AI recommendations: what was useful and what was not

The following choices were checked against the specification and test results. The separate AI notes give a short record of the assistance and alternatives considered.

The suggestion to share quote construction was useful because it addressed actual repetition in the code. Passing the already-loaded state into the helper also avoids loading the file again during placement. The quote/order comparison tests check that the two operations still agree.

Dictionary lookup was used for internal stock changes, but not directly on raw product IDs. A list or dictionary passed as an ID would otherwise cause a TypeError. Those requests should raise ValueError, and the validation tests cover them. With only five products, keeping the existing scan is also simple enough.

Using `round(subtotal * 0.1)` would look shorter, but it would not reliably meet the required rounding rule. Python uses ties-to-even rounding; the assignment requires half cents to round up. The integer formula was kept. A temporary price of 5,005 cents checks that the discount is 501 cents. That price is a test fixture because the normal catalog cannot produce this rounding case.

Reusing standard delivery's free-delivery condition for express was also rejected. It would make expensive express orders free, which directly conflicts with CR-01. The separate express branch is small, and tests check that the fee is neither waived nor discounted.

A database or a class hierarchy for delivery options would be more work than this change needs. The existing JSON storage and a short fulfillment conditional were kept. Duplicate lines were also left separate in the results; only their combined quantity is used for checking stock.

6. Result and limits

All 49 tests pass on Windows with Python 3.12.10. The original four tests are still included. The seven selected mistake checks add confidence, but do not prove that every possible case is correct. No coverage percentage is claimed.

Menu input is scripted, and permission errors, EOF and Ctrl+C are simulated. Python 3.10 was not separately tested. Concurrent writers and recovery from a crash while saving are outside the supplied specification.

The public repository contains the source, tests and development history. To run the checks from the project folder:

```
python -m unittest discover -s tests -v
python scripts/verify.py
```

Sources: the starter README and functional specification, the supplied CR-01, and the test logs under `evidence/`. The original full assignment handout was unavailable, so its exact formatting and grading requirements still need checking.